```
template <class R1, class R2>
class ratio_equal:
    public std::integral_constant<
        bool,(R1::num == R2::num) && (R1::den == R2::den)>
{};
```

Preconditions

`R1` and `R2` must be instantiations of the `std::ratio` class template.

Examples

```
std::ratio_equal<std::ratio<1,3>, std::ratio<2,6> >::value == true
std::ratio_equal<std::ratio<1,3>, std::ratio<1,6> >::value == false
std::ratio_equal<std::ratio<1,3>, std::ratio<2,3> >::value == false
std::ratio_equal<std::ratio<1,3>, std::ratio<1,3> >::value == true
```

D.6.7. std::ratio_not_equal class template

The `std::ratio_not_equal` class template provides a mechanism for comparing two `std::ratio` values for inequality at compile time, using rational arithmetic.

Class definition

```
template <class R1, class R2>
class ratio_not_equal:
    public std::integral_constant<bool, !ratio_equal<R1,R2>::value>
{};
```

Preconditions

`R1` and `R2` must be instantiations of the `std::ratio` class template.

Examples

```
std::ratio_not_equal<std::ratio<1,3>, std::ratio<2,6> >::value ==
false
std::ratio_not_equal<std::ratio<1,3>, std::ratio<1,6> >::value ==
true
std::ratio_not_equal<std::ratio<1,3>, std::ratio<2,3> >::value ==
```